

The `pendingNotify` Mutex: An Isolated Proof

Auxiliary to *Biff Notification System: A Z Specification*

September 2026

1 Purpose

`notification.tex` § *The Race: Three Critical Sections, One Shared Flag* models three critical sections — suspenders, belt, capture/reconnect — each split into a read/decide step and a write step, and proves a mutex serialising them closes a lost-update race on *pendingNotify*. That proof is stated there against the full *NotificationState* schema, which also carries the talk and wall marker machinery (Sections 2–10 of the main document) so that the race schemas can share *NotificationState*’s existing framing conventions. Combined, the reachable space is large enough that an exhaustive check — even at the floor cardinality every carrier in the main document already uses — did not terminate in a reasonable time or memory budget. None of that machinery participates in the race: the race is a five-variable property. This file states it as its own machine and checks it exhaustively.

2 State

$ZBOOL ::= ztrue \mid zfalse$

$RaceSite ::= raceSuspenders \mid raceBelt \mid raceCapture$

RaceState

$pendingNotify : ZBOOL$

$inFlight : \mathbb{P} RaceSite$

$beltSnapshot : ZBOOL$

$capSnapshot : ZBOOL$

$raceLost : ZBOOL$

$notifyLost : ZBOOL$

$\#inFlight \leq 1$

$raceLost = zfalse$

$notifyLost = zfalse$

pendingNotify is the shared flag every site reads and writes. *inFlight* tracks which sites have completed their read/decide step but not yet their write — the gap a non-atomic **await** opens up in the real code. *beltSnapshot* and *capSnapshot* are what each “clearing” site read at the moment it entered that gap; *raceSuspenders* never clears (its only write is an unconditional re-arm, *Suspenders Failure and the No-Recovery Sink* of the main document), so it needs no snapshot. *raceLost* is round 2’s witness: it fires when a clearing site’s eventual write is based on a value that has since changed underneath it. *notifyLost* is round 1’s witness, restated at the current three-site granularity: it fires when a site’s failure commit leaves *pendingNotify* un-re-armed. The invariant $\#inFlight \leq 1$ is the mutex; $raceLost = zfalse$ and $notifyLost = zfalse$ are the two safety properties it and the round 1 fix are meant to establish jointly. All are stated here exactly as they are in `notification.tex`, restricted to the variables that matter.

3 Initialisation

RaceInit

RaceState'

pendingNotify' = *zfalse*

inFlight' = $\emptyset$

beltSnapshot' = *zfalse*

capSnapshot' = *zfalse*

raceLost' = *zfalse*

notifyLost' = *zfalse*

4 Suspenders (set-only)

The suspenders site's only effect on *pendingNotify* is an unconditional re-arm on failure (*notification.tex*'s *PollTickNotifyFail*); its success commit does not touch *pendingNotify* at all and is therefore omitted here — it contributes nothing to reachability of *raceLost* or *notifyLost*. *SusCommitSetUnrecorded* is round 1's defect at this site, restated: the commit fires but leaves *pendingNotify* untouched instead of re-arming it.

SusBegin

Δ *RaceState*

raceSuspenders $\notin$ *inFlight*

inFlight' = *inFlight* $\cup$ {*raceSuspenders*}

pendingNotify' = *pendingNotify*

beltSnapshot' = *beltSnapshot*

capSnapshot' = *capSnapshot*

raceLost' = *raceLost*

notifyLost' = *notifyLost*

SusCommitSet

Δ *RaceState*

raceSuspenders $\in$ *inFlight*

inFlight' = *inFlight* $\setminus$ {*raceSuspenders*}

pendingNotify' = *ztrue*

beltSnapshot' = *beltSnapshot*

capSnapshot' = *capSnapshot*

raceLost' = *raceLost*

notifyLost' = *notifyLost*

SusCommitSetUnrecorded

Δ *RaceState*

raceSuspenders $\in$ *inFlight*

inFlight' = *inFlight* $\setminus$ {*raceSuspenders*}

pendingNotify' = *pendingNotify*

beltSnapshot' = *beltSnapshot*

capSnapshot' = *capSnapshot*

raceLost' = *raceLost*

notifyLost' = *ztrue*

5 Belt

<i>BeltBegin</i>
$\Delta\text{RaceState}$
$\text{raceBelt} \notin \text{inFlight}$ $\text{inFlight}' = \text{inFlight} \cup \{\text{raceBelt}\}$ $\text{beltSnapshot}' = \text{pendingNotify}$ $\text{pendingNotify}' = \text{pendingNotify}$ $\text{capSnapshot}' = \text{capSnapshot}$ $\text{raceLost}' = \text{raceLost}$ $\text{notifyLost}' = \text{notifyLost}$

<i>BeltCommitClear</i>
$\Delta\text{RaceState}$
$\text{raceBelt} \in \text{inFlight}$ $\text{inFlight}' = \text{inFlight} \setminus \{\text{raceBelt}\}$ $\text{pendingNotify}' = \text{zfalse}$ $((\text{pendingNotify} \neq \text{beltSnapshot}) \Rightarrow \text{raceLost}' = \text{ztrue})$ $((\text{pendingNotify} = \text{beltSnapshot}) \Rightarrow \text{raceLost}' = \text{raceLost})$ $\text{beltSnapshot}' = \text{beltSnapshot}$ $\text{capSnapshot}' = \text{capSnapshot}$ $\text{notifyLost}' = \text{notifyLost}$

<i>BeltCommitSet</i>
$\Delta\text{RaceState}$
$\text{raceBelt} \in \text{inFlight}$ $\text{inFlight}' = \text{inFlight} \setminus \{\text{raceBelt}\}$ $\text{pendingNotify}' = \text{ztrue}$ $\text{beltSnapshot}' = \text{beltSnapshot}$ $\text{capSnapshot}' = \text{capSnapshot}$ $\text{raceLost}' = \text{raceLost}$ $\text{notifyLost}' = \text{notifyLost}$

<i>BeltCommitSetUnrecorded</i>
$\Delta\text{RaceState}$
$\text{raceBelt} \in \text{inFlight}$ $\text{inFlight}' = \text{inFlight} \setminus \{\text{raceBelt}\}$ $\text{pendingNotify}' = \text{pendingNotify}$ $\text{beltSnapshot}' = \text{beltSnapshot}$ $\text{capSnapshot}' = \text{capSnapshot}$ $\text{raceLost}' = \text{raceLost}$ $\text{notifyLost}' = \text{ztrue}$

6 Capture/Reconnect Flush

CapBegin's guard $\text{pendingNotify} = \text{ztrue}$ mirrors *CaptureSessionFlushBegin*'s in the main document: capture only enters this critical section when there is something recorded to flush.

<i>CapBegin</i>	
$\Delta RaceState$	
	$raceCapture \notin inFlight$ $pendingNotify = ztrue$ $inFlight' = inFlight \cup \{raceCapture\}$ $capSnapshot' = pendingNotify$ $pendingNotify' = pendingNotify$ $beltSnapshot' = beltSnapshot$ $raceLost' = raceLost$ $notifyLost' = notifyLost$

<i>CapCommitClear</i>	
$\Delta RaceState$	
	$raceCapture \in inFlight$ $inFlight' = inFlight \setminus \{raceCapture\}$ $pendingNotify' = zfalse$ $((pendingNotify \neq capSnapshot) \Rightarrow raceLost' = ztrue)$ $((pendingNotify = capSnapshot) \Rightarrow raceLost' = raceLost)$ $beltSnapshot' = beltSnapshot$ $capSnapshot' = capSnapshot$ $notifyLost' = notifyLost$

<i>CapCommitSet</i>	
$\Delta RaceState$	
	$raceCapture \in inFlight$ $inFlight' = inFlight \setminus \{raceCapture\}$ $pendingNotify' = ztrue$ $beltSnapshot' = beltSnapshot$ $capSnapshot' = capSnapshot$ $raceLost' = raceLost$ $notifyLost' = notifyLost$

<i>CapCommitSetUnrecorded</i>	
$\Delta RaceState$	
	$raceCapture \in inFlight$ $inFlight' = inFlight \setminus \{raceCapture\}$ $pendingNotify' = pendingNotify$ $beltSnapshot' = beltSnapshot$ $capSnapshot' = capSnapshot$ $raceLost' = raceLost$ $notifyLost' = ztrue$

7 Verification

Eleven operations, six state components, $\#inFlight \leq 1$ bounding *inFlight* to four possible values: the whole reachable space is small by construction. Two runs, exactly as in `notification.tex` § *The Race: Three Critical Sections, One Shared Flag*, at `DEFAULT_SETSIZE 2` (there are no carrier-typed components left to scope, so the setsize binds nothing here — it is given only for invocation parity with the main document):

Unsafe: the race, reproduced. All three invariant conjuncts removed from *RaceState* (an operation schema’s implicit $RaceState \wedge RaceState'$ framing means *all* must go—leaving $raceLost = zfalse$ or

notifyLost = *zfalse* in place while dropping only the mutex would still forbid every transition that sets either witness to *ztrue*, hiding the defect instead of exposing it, exactly as *notifyLost* = *zfalse* had to be dropped whole in `notification.tex` § *Suspenders Failure and the No-Recovery Sink*:

```
probccli docs/notification-race.tex -model_check -noltl -coverage \
  -goal "raceLost = ztrue" \
  -p DEFAULT_SETSIZE 2 -p MAX_OPERATIONS 2000

Found GOAL in state id 15
States analysed: 30   Transitions fired: 129
*** COUNTER EXAMPLE FOUND ***
*** TRACE (length=5):
  1: INITIALISATION(beltSnapshot=zfalse,capSnapshot=zfalse,inFlight={},
    notifyLost=zfalse,pendingNotify=zfalse,raceLost=zfalse)
  2: SusBegin
  3: BeltBegin
  4: SusCommitSet
  5: BeltCommitClear(inFlight={},pendingNotify=zfalse,raceLost=ztrue)
```

This is precisely the belt/suspenders interleaving described informally in `notification.tex` § *The Race: Three Critical Sections, One Shared Flag*: both sites read/decide before either writes (steps 2–3; belt’s read of *pendingNotify* = *zfalse* decides “clear”); suspenders commits first, *pendingNotify*’ = *ztrue* (step 4), while belt’s write is still outstanding; belt’s commit (step 5) then writes *zfalse* over it, using the decision it made before the re-arm existed. *pendingNotify* ends *zfalse* and *raceLost* records that the write was stale when it landed.

The same file, same goal search technique, reproduces round 1’s defect at the new three-site granularity in two steps:

```
probccli docs/notification-race.tex -model_check -noltl -coverage \
  -goal "notifyLost = ztrue" \
  -p DEFAULT_SETSIZE 2 -p MAX_OPERATIONS 2000

Found GOAL in state id 5
States analysed: 2   Transitions fired: 9
*** COUNTER EXAMPLE FOUND ***
*** TRACE (length=3):
  1: INITIALISATION(beltSnapshot=zfalse,capSnapshot=zfalse,inFlight={},
    notifyLost=zfalse,pendingNotify=zfalse,raceLost=zfalse)
  2: BeltBegin
  3: BeltCommitSetUnrecorded
```

A belt send fails, and the commit that should re-arm *pendingNotify* leaves it untouched instead: two steps, no interleaving required—this is round 1’s defect, not round 2’s, restated for belt rather than suspenders and reachable on its own.

Safe: the mutex plus the re-arm, exhaustive. *RaceState* as committed above ($\#inFlight \leq 1$, *raceLost* = *zfalse*, and *notifyLost* = *zfalse* all present):

```
probccli docs/notification-race.tex -model_check -noltl -coverage \
  -p DEFAULT_SETSIZE 2 -p MAX_OPERATIONS 2000

Model checking finished, all open states visited
Model checking time: 33 ms   States analysed: 22   Transitions fired: 41
No counter example found. ALL states visited.
Coverage: [STATES (23), deadlocked:0, invariant_violated:0,
  invariant_not_checked:0, open:0, COVERED_OPERATIONS (9): BeltBegin,
  BeltCommitClear, BeltCommitSet, CapBegin, CapCommitClear,
  CapCommitSet, INITIALISATION, SusBegin, SusCommitSet,
  UNCOVERED_OPERATIONS (3): BeltCommitSetUnrecorded,
  CapCommitSetUnrecorded, SusCommitSetUnrecorded]
```

Twenty-three states, complete, in 33 ms. The three “Unrecorded” defect operations—round 1’s finding, now stated and proved dead at all three sites in one pass—are reported uncovered, the same proof shape as *WithdrawStale* in the main document. *raceLost = zfalse* and *notifyLost = zfalse* both hold across the entire reachable space. These results are cited from `notification.tex` § *The Race: Three Critical Sections, One Shared Flag* alongside that document’s own full-model goal-directed counterexample (the same interleaving, found via a different pair of colliding sites because ProB’s BFS explores the larger model in a different order); this file exists so the *exhaustive* half of the claim rests on a check that terminates in milliseconds, not on the full model’s combinatorial product with unrelated marker machinery, which did not terminate in a reasonable time or memory budget when tried directly.